

Kotlin のすべての変数やデータ構造には型があります。型は、その変数やデータ構造に対してどのような操作が可能かをコンパイラに伝えるため、重要です。言い換えれば、それらがどのような関数やプロパティを持っているかを示すものです。

前回の章では、前の例において、Kotlin は `customers` の型が `Int` であることを判別できました。Kotlin が型を推論する能力は、**型推論**と呼ばれます。 `customers` には整数値が代入されます。これに基づき、Kotlin は `customers` の型が数値型 `Int` であると推論します。 その結果、コンパイラは `customers` に対して算術演算を実行できることを認識します：

```
fun main() {
//sampleStart
    var customers = 10

    // Some customers leave the queue
    customers = 8

    customers = customers + 3 // Example of addition: 11
    customers += 7           // Example of addition: 18
    customers -= 3           // Example of subtraction: 15
    customers *= 2           // Example of multiplication: 30
    customers /= 3           // Example of division: 10

    println(customers) // 10
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-basic-types-arithmetic"}

`+=`、`-=`、`*=`、`/=`、および `%=` は、拡張代入演算子です。詳細については、[拡張代入](#) を参照してください。

{style="tip"}

Kotlin には、全部で以下の基本型があります： で以下の基本型があります：

カテゴリ	基本型	使用例 code
Integers	Byte, Short, Int, Long	<code>val year: Int = 2020</code> <code>val amount: Long = 350_000_000</code>
Unsigned integers	UByte, UShort, UInt, ULong	<code>val score: UInt = 100u</code>
Floating-point numbers	Float, Double	<code>val currentTemp: Float = 24.5f</code> <code>val price: Double = 19.99</code>
Booleans	Boolean	<code>val isEnabled: Boolean = true</code>
Characters	Char	<code>val separator: Char = ','</code>
Strings	String	<code>val message: String = "Hello, world!"</code>

基本型とその特性に関する詳細については、[型の概要](#)を参照してください。

この知識があれば、変数を宣言しておき、後で初期化することができます。Kotlinでは、変数が最初に読み込まれる前に初期化されていれば、これを適切に処理してくれます。

変数を初期化せずに宣言するには、その型を `:` で指定します。例えば：

```
fun main() {
//sampleStart
    // 初期化なしで宣言された変数
    val d: Int
    // 変数が初期化されました
    d = 3

    // 変数が明示的に型指定され、初期化されている
    val e: String = "hello"

    // 変数は初期化されているため、読み取ることができます
    println(d) // 3
    println(e) // hello
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-basic-types-initialization"}

変数を読み込む前に初期化しないと、次のようなエラーが表示されます：

```
fun main() {
//sampleStart
    // 初期化なしで宣言された変数
    val d: Int

    // エラーを発生する
    println(d)
    // 変数'd'を初期化する必要があります
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-basic-types-no-initialization" validate="false"}

基本的な型の宣言方法を学んだところで、次は[コレクション](#)について学びましょう。

Practice {completion-point="true"}

演習 {completion-point="true"}

Exercise {initial-collapse-state="collapsed" collapsible="true"}

課題 {initial-collapse-state="collapsed" collapsible="true"}

各変数について、正しい型を明示的に宣言してください：

|---|---|

```
fun main() {  
    val a: Int = 1000  
    val b = "log message"  
    val c = 3.14  
    val d = 100_000_000_000_000  
    val e = false  
    val f = '\n'  
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-basic-types-exercise"}

|---|---|

```
fun main() {  
    val a: Int = 1000  
    val b: String = "log message"  
    val c: Double = 3.14  
    val d: Long = 100_000_000_000_000  
    val e: Boolean = false  
    val f: Char = '\n'  
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-basic-types-solution"}

- [Previous step](#)
- [Next step](#)